algorithms homework 3

mahmood

January 16, 2023

homework on recursion

note

please do provide feedback on mistakes you find, if any

Question 1. write a recursive algorithm that finds the 2 biggest numbers in a given array that halves the array on each recursive call

Answer 1.0.1.

```
BIGGEST-TWO(A):
    if SIZE(A) = 1 then
        return A[1], −∞
    if SIZE(A) = 2 then
        return A[1], A[2]
    left_1, left_2 ← BIGGEST-TWO(A[1, ..., SIZE(A)/2])
    right_1, right_2 ← BIGGEST-TWO(A[SIZE(A)/2 + 1, ..., SIZE(A)])
    biggest_1 = MAX({left_1, left_2, right_1, right_2})
    biggest_2 = MAX({left_1, left_2, right_1, right_2} − {biggest_1})
    return biggest_1, biggest_2
```

python version of the algorithm:

```python
def biggest_two(arr):
    if len(arr) == 1:
        return arr[0], -1000
    if len(arr) == 2:
        return arr[0], arr[1]
    left_1, left_2 = biggest_two(arr[0:len(arr)//2])
    right_1, right_2 = biggest_two(arr[len(arr)//2:len(arr)])
    biggest_1 = max({left_1, left_2, right_1, right_2})
    biggest_2 = max({left_1, left_2, right_1, right_2} - {biggest_1})
    return biggest_1, biggest_2

print(biggest_two([10, 20, 30, 35, 2, 17]))
```

```
(35, 30)
```

Subquestion 1.1. find the time complexity of the algorithm

Answer 1.1.1. we consider the recurrence relation $T(n) = 2T\left(\frac{n}{2}\right) + C$:

$$
\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + C \\
T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{4}\right) + C \\
T\left(\frac{n}{4}\right) &= 2T\left(\frac{n}{8}\right) + C \\
T\left(\frac{n}{2^{\lg n}}\right) &= 2T\left(\frac{n}{2^{\lg n+1}}\right) + C \\
T(n) &= 2\left(2T\left(\frac{n}{4}\right) + C\right) + C = 4T\left(\frac{n}{4}\right) + 3C \\
&= 4\left(2T\left(\frac{n}{8}\right) + C\right) + 3C = 8T\left(\frac{n}{8}\right) + 7C \\
&= 8\left(2T\left(\frac{n}{16}\right) + C\right) + 7C = 16T\left(\frac{n}{16}\right) + 15C \\
&\vdots \\
&= 2^{\lg n}T\left(\frac{n}{2^{\lg n}}\right) + \sum_{i=0}^{\lg n} 2^i C \\
&= 2^{\lg n}C_1 + \sum_{i=0}^{\lg n} 2^i C \\
&= 2^{\lg n}C_1 + \frac{1 - 2^{\lg(n)+1}}{-1}C \\
&= 2^{\lg n}C_1 - C + 2^{\lg(n)+1} \\
&\Rightarrow \Theta\left(2^{\lg n}\right) = \Theta(n)
\end{aligned}
$$

note that although it doesnt matter, C_1 differs a from C as $C_1 = T(1)$ is the constant time it takes for the base case calls of the recursion

Question 2. find the time complexity of the following algorithm

```
WAI(A, N):
    i ← 1
    while i ≤ N do
        j ← i
        while A[j] = A[j + 1] and (j < N) do
            j++
        j++
        i ← j
```

Answer 2.0.1. at first glance this seems like a $\Theta(n^2)$ algorithm but its time complexity is actually $\Theta(n)$
we track the value of the variable `i`: on each iteration in the main loop, the variable `i` gets incremented as many times as the inner loop runs plus one (`j` is assigned back to `i` so we're basically acting on `i` itself), `i` can be at most incremented `N` times before both loops wouldnt execute anymore, and since on each iteration of either loop i gets incremented both loops are bound by `N` iterations at most so the total time complexity is $\Theta(N)$

Question 3. find the time complexity of the following algorithm

```
PUZZLE(A, N):
    for i ← 1 to N do
        j ← i + 1
        while A[j] ≠ A[i] and j < N do
            A[j] ← A[i]
            j++
```

on the first iteration the inner loop runs and sets all the elements of the array to the value `A[i]` and so the `while` loop would never run again on next iterations of the `for` loop, so for `N-1` iterations of the `for` loop we're doing constant work (all except the first iteration) which amounts to a time complexity of $\Theta(N)$
adding the $\Theta(N)$ from the first iteration we get $\Theta(N) + \Theta(N) \Rightarrow \Theta(N)$
so the time complexity is $\Theta(N)$

Question 4. given a function `P(A,l,k)` that sorts `k` elements in `A` starting at index `l`
the algorithm runs as follows:

1. if k<10, apply bubble sort

2. run `P(A, l, 3k/4)`

3. run `P(A, l+k/4, 3k/4)`

4. run `P(A, l, 3k/4)`

Subquestion 4.1. prove/disprove whether the algorithm actually sorts

Answer 4.1.1. we prove the algorithm using perfect induction:
for all $k < 10$, the algorithm indeed works as we know bubble sort works
we assume that the algorithm works for all $n > k \geq 10$
we assume descending order of sorting **without loss of generality**
we prove the algorithm works for $k = n$:
when $n = k$ this is what the algorithm does:

1. $A\left[l, \ldots, l + \frac{3n}{4} - 1\right]$ is sorted

2. $A\left[l + \frac{n}{4}, \ldots, l + n - 1\right]$ is sorted
   before this second sorting operation, we knew for sure that the elements in the first fourth of the array are all smaller than the elements in the following 2/4's of the array because this 3/4 subarray as a whole is sorted
   after the sorting operation the last fourth of the array would be merged into the 2/4's in the middle resulting in the last 3/4's in the array being sorted, now we knew before that 2/4's of the elements in the array are bigger than the elements in the first fourth, so after merging the last fourth we now know that there are **atleast** 2/4's that are bigger than the first fourth:

   $$\left(\forall x \in A\left[l, \ldots, l + \frac{n}{4} - 1\right]\right)\left(\forall y \in A\left[l + \frac{2n}{4}, \ldots, l + n - 1\right]\right)[x \geq y]$$

   in words: each element in the last 2/4's of the array is bigger than all elements in the first fourth of the array

3. sort $A\left[l, \ldots, l + \frac{3k}{4} - 1\right]$
   a consequence of the third sorting step is that the first fourth would be merged into the the second 3/4's and its elements would be spread across the first 2/4's, we know the last 2/4's wouldnt be affected by this sorting operation because they are all for sure bigger than the first fourth
   and so the array would be for sure sorted after the third step.

and since we've proved that the algorithm is correct for $k = n$ we've proved its correctness by perfect induction

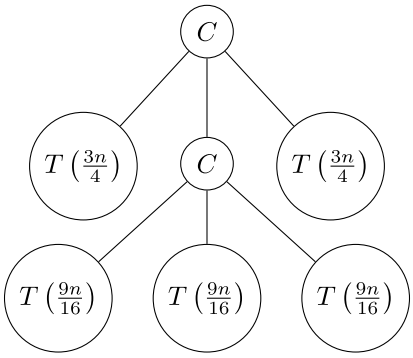
Subquestion 4.2. find the time complexity of the algorithm using recursion trees, find the number of nodes in each level and the number of levels

> **note**
> im almost sure there is something wrong here but im not sure how to improve this solution

Answer 4.2.1. we propose the following recurrence relation:

$$T(n) = 3T\left(\frac{3}{4}n\right) + C$$

and build a recursion tree from it:



let x denote the height of the tree, we know $\left(\frac{3}{4}\right)^x n < 10$ because $n < 10$ is the base case
the number of nodes in each level y is 3^{y-1}
according to the pattern in the tree:

$$T(n) = \sum_{y=0}^{x-1} 3^y C$$

we might have to check the time complexity for each value in $T(1), \ldots, T(9)$ assigned as the base case only if they do play a role in the total time complexity
the time complexity is the sum of all the nodes in the tree:

$$\left(\frac{3}{4}\right)^x n = 9$$
$$\ln\left(\left(\frac{3}{4}\right)^x n\right) = \ln 9$$
$$\ln n + \ln\left(\frac{3}{4}\right)^x = \ln 9$$
$$x = \frac{\ln 9 - \ln n}{\ln\left(\frac{3}{4}\right)}$$
$$T(n) = \sum_{y=0}^{x-1} 3^y C = \left(\frac{3^x}{2} - \frac{1}{2}\right) C = \left(\frac{3^{\frac{\ln 9 - \ln n}{\ln\left(\frac{3}{4}\right)}}}{2} - \frac{1}{2}\right) C$$
$$\Theta\left(\left(\frac{3^{\frac{\ln 9 - \ln n}{\ln\left(\frac{3}{4}\right)}}}{2} - \frac{1}{2}\right) C\right) \implies \Theta\left(3^{\ln n}\right) \implies \Theta(3^n)$$

note

it confused me for a good while that we have $-\ln n$ and i thought that the expression would converge as n goes to infinity but then i realized that $\ln \frac{3}{4}$ is negative and it would cancel the minus sign so the time would actually diverge

we can notice that the constant 9 played no part in the equation so the result would be the same for all constants 1 to 9
so the time complexity is $\Theta\left(3^n\right)$
the height of the tree is dependant on the initial value of n but i approximate it to be $\frac{\ln 9 - \ln n}{\ln\left(\frac{3}{4}\right)}$

Question 5. modify the `merge` algorithm such that it merges without using elements that have a value of ∞

Answer 5.0.1.

```
MERGE(A, B):
    ap ← bp ← cp ← 1  // pointers for each of the arrays A,B,C
    while cp < m + n + 1 do
        if ap > SIZE(A) and bp > SIZE(B) then
            break
        if ap > SIZE(A) then
            C[cp] ← B[bp]
            bp++
        else if bp > SIZE(B) then
            C[cp] ← A[ap]
            ap++
        else if A[ap] ≤ B[bp] then
            C[cp] ← A[ap]
            ap++
        else
            C[cp] ← B[bp]
            bp++
        cp++
    return C
```